PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

for

ChatSphere – Real-Time Direct & Group Messaging Platform

Version 1.1 approved

Prepared by Team 16 – Satvik L, Prithvi Aithal, Sourish Ganesh
PES University, Bangalore – Dept. of CSE, Section G

Team Members and SRNs

Sl.	Student Name	SRN	USN
1	Satvik L	PES1UG24CS423	PES1202400676
2	Prithvi Aithal	PES1UG24CS909	PES1202400699
3	Sourish Ganesh	PES1UG25CS847	PES1202503600

September 2026

Revision History

Name	Date	Reason For Changes	Version
Team 16	Sep 2026	Initial SRS baseline for Requirements submission	1.0
Team 16	Sep 2026	Added security objectives and expanded security requirements	1.1

Table of Contents

Revision History 1
Table of Contents 1
1. Introduction 1
1.1 Purpose 1
1.2 Intended Audience and Reading Suggestions 1
1.3 Product Scope 1
1.4 References 1
2. Overall Description 1
2.1 Product Perspective 1
2.2 Product Functions 1
2.3 User Classes and Characteristics 1
2.4 Operating Environment 1
2.5 Design and Implementation Constraints 1
2.6 Assumptions and Dependencies 1
3. External Interface Requirements 1
3.1 User Interfaces 1
3.2 Software Interfaces 1
3.3 Communications Interfaces 1
4. Analysis Models 1
5. System Features 1
5.1 Authentication & Presence Management 1
5.2 Direct Messaging 1
5.3 Group / Room Chat 1
5.4 Message History & Status Tracking 1
5.5 Room Administration 1
6. Other Nonfunctional Requirements 1
6.1 Performance Requirements 1
6.2 Safety Requirements 1
6.3 Security Objectives and Requirements 1
6.4 Software Quality Attributes 1
6.5 Business Rules 1
7. Other Requirements 1
8. Validation Specification – Initial Planning 1
9. Project Plan 1
9.1 Work Breakdown Structure (WBS) 1
9.2 Schedule (Sprint / Gantt Summary) 1
9.3 Tool Stack 1
Appendix A: Glossary 1
Appendix B: Field Layouts 1
Appendix C: Requirement Traceability Matrix 1

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of ChatSphere, a real-time direct and group messaging web application built as the Level 3 Software Engineering mini-project (UE24CS341A) by Team 16. It covers the complete set of functional, non-functional and security requirements for the initial release and is the baseline against which the system's design, implementation and testing will be validated.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for: the development team (Satvik L, Prithvi Aithal, Sourish Ganesh), who use it as the contract for design and implementation; the course faculty/evaluators, who use it to assess requirement quality and traceability; and the team members acting as testers, who derive test cases directly from Sections 5 and 6. Readers unfamiliar with the domain should first read Section 2 (Overall Description) before the detailed feature requirements in Section 5.

1.3 Product Scope

ChatSphere provides registered users with low-latency, one-to-one and group (room-based) text messaging over a persistent WebSocket connection, including real-time presence, typing indicators, delivery/read status and persisted, searchable message history. Its purpose is both to give a small community (e.g. a project team or campus group) a self-hostable communication tool, and to demonstrate the team's understanding of socket programming, event-driven architecture and the full software development life cycle.

1.4 References

- PES University, Dept. of CSE – “Jackfruit – Software Engineering Project Guidelines 2026” (UE24CS341A).
- PES University – “Software Engineering – Level 3 Student Submission Guide & Complete Exemplar” (Coffee Kiosk).
- Karl Wiegers – IEEE-style Software Requirements Specification template (structure followed by this document).
- Socket.IO official documentation – https://socket.io/docs/
- Express.js official documentation – https://expressjs.com/

2. Overall Description

2.1 Product Perspective

ChatSphere is a new, self-contained, three-tier web application; it is not a follow-on to an existing system. It consists of a React single-page-application client, a Node.js/Express + Socket.IO server that exposes both REST endpoints (for auth and history retrieval) and a WebSocket gateway (for real-time events), and a persistence tier composed of MongoDB (durable data – users, rooms, messages) and Redis (ephemeral presence/session state). The high-level flow is: Client ↔ REST API / WebSocket Gateway ↔ Service Layer ↔ MongoDB / Redis.

2.2 Product Functions

- User registration and secure (JWT-based) login.
- Real-time one-to-one direct messaging.
- Room creation, joining and leaving for group chat.
- Real-time group messaging within a room.
- Online/offline presence broadcasting.
- Real-time typing indicators.
- Persistent, paginated message history retrieval.

- Room administration – promoting or removing a member.
- Message delivery/read status tracking.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Registered User	Register/login; send & receive direct and group messages; create/join/leave rooms; view own conversation history; view presence	Non-technical, expects near-instant delivery; frequent short sessions from browser
Room Admin	All Registered User privileges, plus: promote another member to admin, remove a member from a room they administer	Same technical profile as Registered User; privilege is scoped only to rooms they created or were promoted in

2.4 Operating Environment

- Client: any evergreen desktop browser with WebSocket support (Chrome, Firefox, Edge).
- Server: Node.js 20 LTS runtime on Linux, packaged as a Docker container.
- Database: MongoDB 6+; Redis 7+ for presence/session pub-sub.
- Network: HTTPS for REST calls, WSS (WebSocket Secure) for the real-time channel.

2.5 Design and Implementation Constraints

- Must use WebSocket-based real-time communication (Socket.IO) per the course assignment – polling-only designs are not acceptable.
- Agile methodology is mandatory; work must be tracked as user stories/sprints on GitHub Projects (or Jira).
- Git/GitHub is the system of record for version control; individual Git commits are checked as evidence of contribution.
- A CI/CD pipeline, code reviews and security validation are mandatory course deliverables for later phases.
- Team of exactly 3 members; each member must own one complete functional feature end-to-end (Section 9 of the Synopsis).

2.6 Assumptions and Dependencies

- Users have a reasonably stable internet connection; brief disconnects (under ~60s) are expected to resume without message loss, but ChatSphere does not guarantee delivery over extended offline periods beyond queued-message retry.
- The project depends on third-party open-source libraries (Express, Socket.IO, Mongoose, JWT, bcrypt) remaining available and API-stable for the duration of the project.
- This phase targets a single-server deployment; multi-instance/horizontal scaling of the Socket.IO layer is out of scope (see Synopsis Section 7).

3. External Interface Requirements

3.1 User Interfaces

- Registration / Login screen – username, email, password fields, client-side validation and error messaging.
- Chat Dashboard – left panel listing direct conversations and joined rooms with unread counts and presence dots; right panel showing the active conversation.
- Message Composer – text input with a live typing indicator broadcast to the other participant(s), and per-message delivery/read ticks.
- Room Panel – member list with roles, and (for a Room Admin) controls to promote or remove a member.
- Search – a search box to find existing users or rooms to start a new conversation.

3.2 Software Interfaces

- REST API (JSON over HTTPS) – used for registration, login, and paginated history retrieval; implemented with Express.js.
- WebSocket API (Socket.IO over WSS) – used for all real-time events: message send/receive, typing, presence, room join/leave, room-admin actions.
- MongoDB via the Mongoose ODM – durable storage for User, Room, RoomMembership and Message documents.
- Redis – pub/sub channel for presence propagation and Socket.IO adapter for cross-instance event broadcast (forward-compatibility for future scaling).

3.3 Communications Interfaces

- All REST traffic is served over HTTPS; all real-time traffic is served over WSS (WebSocket Secure).
- Messages exchanged between client and server are JSON-encoded.
- Authentication uses JSON Web Tokens (JWT) sent as a Bearer token on REST calls and as an auth payload during the Socket.IO handshake.

4. Analysis Models

The use case diagram below identifies the two actors (Registered User and the more privileged Room Admin) and the major use cases they participate in.

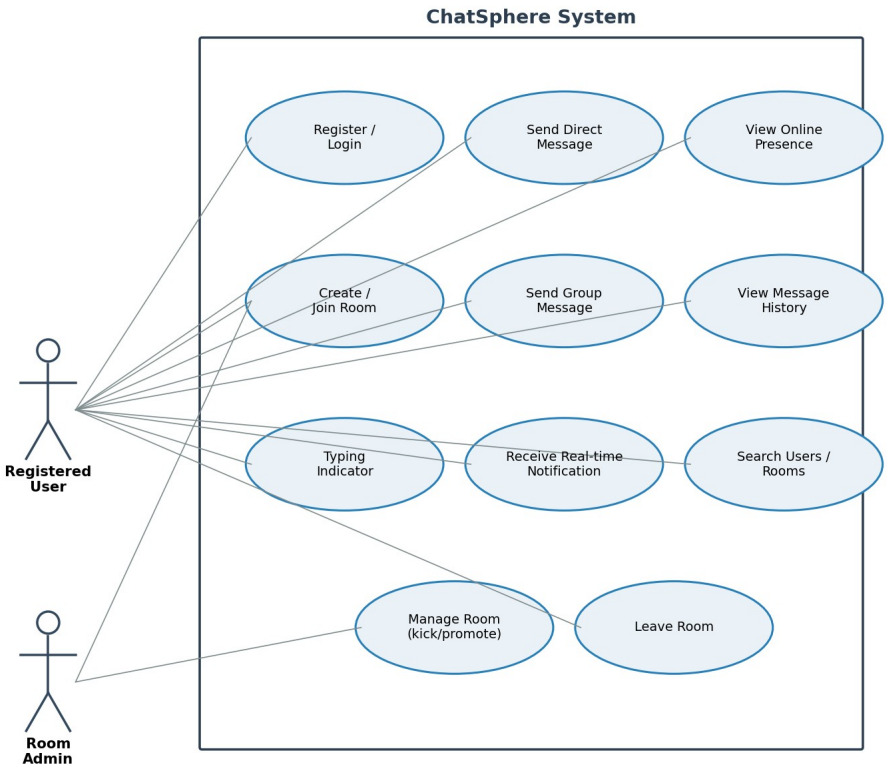


Figure 1 – Use Case Diagram for ChatSphere

The entity-relationship model below shows the core persisted entities – User, Room, RoomMembership (the join entity carrying each member's role) and Message – plus the ephemeral Session entity used to map a live socket connection back to a user.

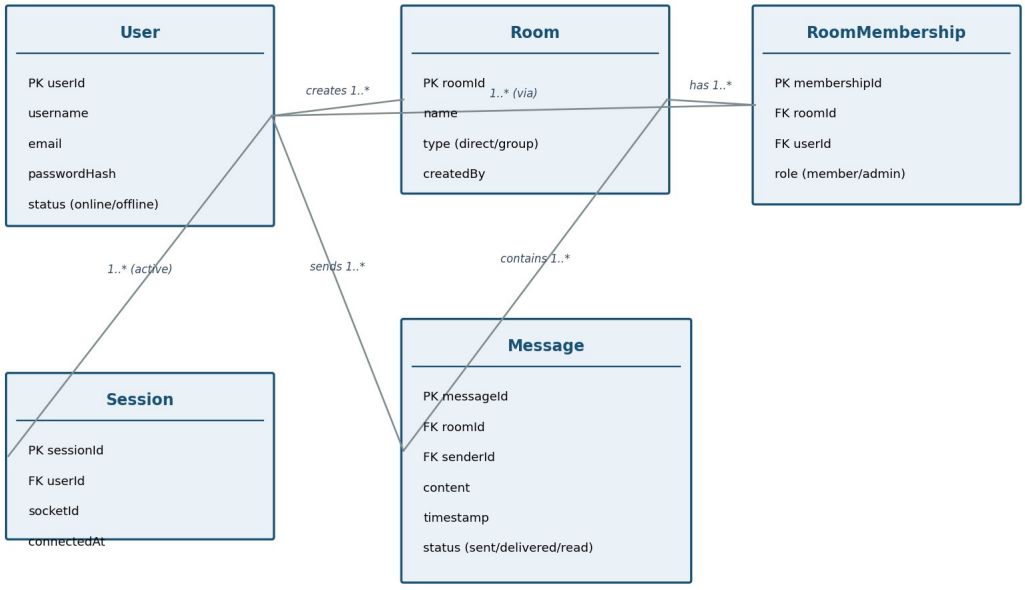


Figure 2 – Entity-Relationship Diagram for ChatSphere's data model

Use Cases

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Registered User	Application reachable	Authenticated session (JWT) issued
UC-02	Send Direct Message	Registered User	Logged in	Message delivered (or queued if recipient offline)
UC-03	Create / Join Room	Registered User	Logged in	Room membership created
UC-04	Send Group Message	Registered User	Room member	Message broadcast to all online room members
UC-05	View Message History	Registered User	Conversation/room exists	Paginated history returned
UC-06	Manage Room	Room Admin	Admin authenticated	Member role updated / member removed
UC-07	View Presence	Registered User	Logged in	Online/offline status of contacts shown

5. System Features

Requirements are organized by system feature. Each feature lists a description/priority, its stimulus/response behaviour, and its detailed, uniquely-tagged functional requirements. Each requirement is concise, unambiguous, verifiable and necessary, per the course's requirement-writing rule.

5.1 Authentication & Presence Management

Description and Priority

Lets a person register a ChatSphere account, log in to obtain a session, and lets the system reflect that user's connection state to others in real time. Priority: High – every other feature depends on it. Owned by Sourish Ganesh.

Stimulus/Response Sequences

- User submits the registration form → system creates the account or returns a validation error.
- User submits valid credentials → system returns a JWT and the client opens an authenticated socket connection.
- Client's socket connects/disconnects → system broadcasts the user's presence change to relevant contacts/rooms.

Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a new user to register using a unique username and a unique email address.	High	Functional test
FR-002	The system shall hash user passwords (bcrypt) before storage; plaintext passwords shall never be persisted.	High	Code/config review
FR-003	The system shall allow a registered user to log in with valid credentials and receive a signed JWT session token.	High	Functional test
FR-004	The system shall authenticate the WebSocket handshake using the JWT issued at login and reject the connection if the token is missing, expired or invalid.	High	Security test
FR-005	The system shall broadcast a user's online status to their direct contacts and room members immediately upon socket connection.	High	Integration test
FR-006	The system shall broadcast a user's offline status to their direct contacts and room members within 5 seconds of socket disconnection.	Medium	Integration test

5.2 Direct Messaging

Description and Priority

Lets any two registered users exchange private, real-time text messages with typing and delivery-status feedback. Priority: High. Owned by Prithvi Aithal.

Stimulus/Response Sequences

- User selects a contact and sends a message → system delivers it instantly if the recipient is online, otherwise queues it.
- User starts typing in an open conversation → system notifies the other participant in real time; stops after a short idle period.
- Recipient's client acknowledges a message → system updates that message's status for the sender.

Functional Requirements

ID	Requirement	Priority	Verification
FR-007	The system shall allow an authenticated user to send a direct text message to another registered user.	High	Functional test
FR-008	The system shall deliver a direct message to the recipient's client in real time over WebSocket when the recipient is online.	High	Integration test
FR-009	The system shall persist a direct message addressed to an offline recipient and deliver it automatically the next time that recipient connects.	High	Integration test

FR-010	The system shall emit a typing-indicator event to the other participant while the sender is actively composing a message, and clear it after a short idle timeout.	Medium	Functional test
FR-011	The system shall track and expose each direct message's status as one of: sent, delivered, or read.	High	Unit + integration test

5.3 Group / Room Chat

Description and Priority

Lets users organize into named rooms and exchange group messages broadcast to every connected member. Priority: High. Owned by Satvik L.

Stimulus/Response Sequences

- User requests to create a room with a unique name → system creates the room and makes the creator its first admin.
- User requests to join/leave a room → system updates room membership and notifies existing members.
- A room member sends a message → system broadcasts it to every other currently-connected member of that room.

Functional Requirements

ID	Requirement	Priority	Verification
FR-012	The system shall allow an authenticated user to create a chat room with a unique room name, becoming that room's first admin.	High	Functional test
FR-013	The system shall allow an authenticated user to join an existing room or leave a room they currently belong to.	High	Functional test
FR-014	The system shall broadcast a group message in real time to every member of that room who is currently connected.	High	Integration test
FR-015	The system shall allow only current members of a room to view or send messages within that room.	High	Negative/security test
FR-016	The system shall require every room to retain at least one admin at all times, rejecting an action that would leave it with none.	Medium	Negative test

5.4 Message History & Status Tracking

Description and Priority

Persists every message and exposes it back to clients as searchable, paginated history so conversations survive reconnects and logouts. Priority: High/Medium. Jointly supported by Prithvi Aithal (direct) and Satvik L (group).

Functional Requirements

ID	Requirement	Priority	Verification
FR-017	The system shall persist every direct and group message with its sender, timestamp, and conversation/room reference.	High	Integration test
FR-018	The system shall allow a user to retrieve the message history of a conversation or room in reverse-chronological, paginated order.	High	Functional test

5.5 Room Administration

Description and Priority

Gives a room's admin(s) control over who belongs to the room and who else holds admin rights. Priority: Medium. Owned by Satvik L.

Functional Requirements

ID	Requirement	Priority	Verification
FR-019	The system shall allow a room admin to promote another current member of that room to admin.	Medium	Functional test
FR-020	The system shall allow a room admin to remove a member from that room.	Medium	Functional test
FR-021	The system shall reject a room-administration action attempted by a member who does not hold the admin role for that room.	High	Authorization test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of chat messages shall be delivered to an online recipient within 300 ms under the defined test load.	Latency/load test
NFR-002	Performance	95% of REST API responses (login, history) shall complete within 1 second under the defined test load.	Performance test

6.2 Safety Requirements

ChatSphere is a text-communication tool with no physical actuation, so conventional safety-of-life requirements do not apply. The only safety-adjacent concern is data safety: the system shall persist a message to the database before acknowledging it as ‘sent’ to the sender, so that a server crash immediately after send cannot silently lose an already-acknowledged message.

6.3 Security Objectives and Requirements

Security Objectives

These objectives state the security goals that the requirements below are designed to satisfy; each requirement traces to at least one objective.

- SEC-OBJ-1 (Confidentiality of credentials): protect user credentials and session tokens from disclosure, both at rest (password hashes, never plaintext) and in transit (HTTPS/WSS only), so that an attacker who observes traffic or accesses the database cannot recover a usable password or forge a session.
- SEC-OBJ-2 (Authorization integrity): ensure every direct message, room message and room-administration action is only ever performed by, or visible to, a user who is authenticated and holds the correct role (room member or room admin) for that action, preventing privilege escalation and cross-room data leakage.
- SEC-OBJ-3 (Input and content safety): ensure no user-supplied content (messages, usernames, room names) can be used to inject executable script (XSS) or otherwise corrupt data integrity for other users of the system.
- SEC-OBJ-4 (Availability under abuse): ensure a single connected client cannot degrade service for other users through message flooding or connection abuse, and that security-relevant events are observable after the fact via logging.

Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall never be stored in plaintext; only bcrypt password hashes shall be persisted.	Code/configuration review

SEC-002	Role-based access control shall prevent a non-admin room member from performing room-administration actions.	Authorization tests
SEC-003	All user-supplied message content shall be sanitized/escaped before storage and before rendering, to prevent stored XSS.	Negative/security tests
SEC-004	REST endpoints and WebSocket connections shall reject expired or invalid JWTs.	Security test
SEC-005	The system shall rate-limit message-send events per connection to mitigate spam/flood abuse.	Load/security test
SEC-006	Security-relevant events (failed login attempts, unauthorized room-admin attempts) shall be logged.	System test

6.4 Software Quality Attributes

- Maintainability – business logic is separated into independent, unit-tested services (auth, messaging, room, presence).
- Reliability – a client that reconnects within 60 seconds of a dropped socket resumes without losing any queued message.
- Usability – a new user can register and send their first message within 3 primary screens.
- Testability – core socket event handlers expose a testable interface independent of the live Socket.IO transport, so they can be unit tested with mocked sockets.
- Portability – the server is packaged as a Docker image and configured entirely via environment variables.

6.5 Business Rules

- Only current members of a room may send or read messages in that room.
- Only a room's admin may promote/demote members or remove a member from that room.
- A direct-message conversation exists only between two distinct registered users.
- A password must satisfy a minimum complexity rule (at least 8 characters) before registration succeeds.
- A room must retain at least one admin at all times.

Also include Domain requirements here: message ordering within a single conversation/room must be strictly timestamp-consistent so history always renders in the order messages were actually sent.

7. Other Requirements

- Database: message history is retained indefinitely in this phase; a future release may add user-initiated deletion/retention limits.
- Legal/localization: no legal or localization requirements apply to this academic-project phase; UI text is English-only.
- Reuse: the project reuses open-source libraries (Express, Socket.IO, Mongoose, JWT, bcrypt, Redis client) under their respective permissive (MIT-style) licenses; no proprietary third-party components are used.

8. Validation Specification – Initial Planning

At the Requirements stage, this validation specification is planning-focused: it explains how each category of requirement above will eventually be validated, including planned security/penetration-style testing. It is not the final test report — concrete pass/fail results and the completed Requirement Traceability Matrix (Appendix C) will be produced once Deliverables 3 and 4 (Design, Implementation & Testing) are complete.

Requirement Category	Planned Validation Approach	Example	Timing / Owner
Functional	Black-box functional tests derived from	Verify FR-008: message	Sprints 2–4, feature

	each FR in Section 5	delivered in real time when recipient is online	owner
Security	Authentication/authorization tests and negative/security tests derived from Section 6.3	Verify SEC-002: non-admin member is rejected from room-admin actions	Sprint 5, Sourish Ganesh
Performance	Load/latency testing against the NFR-001/NFR-002 targets	Confirm 95% message delivery within 300 ms under defined concurrent load	Sprint 5, all members
Usability	Task-based walkthrough by a first-time user	New user registers and sends a first message within 3 primary screens	Sprint 4, all members
Reliability	Failure/retry and reconnect scenario testing	Client reconnecting within 60 s resumes without losing a queued message	Sprints 4–5, all members
Unit	Automated unit tests with coverage measurement per module (auth, messaging, room, presence)	Message status transition logic (sent → delivered → read)	Sprints 2–5, all members

Planned penetration-style checks: manual attempts to call room-admin REST/WebSocket actions with a non-admin JWT, an expired JWT and a tampered JWT, plus a check that message content is escaped before storage/render to rule out stored XSS. Results will be recorded in the System Validation Report at Deliverable 5.

9. Project Plan

The project follows an Agile (Scrum-style), sprint-based lifecycle across six two-week sprints, chosen because the feature set (auth, direct messaging, room chat, presence, admin) decomposes naturally into independently demoable increments that map to Section 10 of the Synopsis. Work is tracked as user stories/tasks on GitHub Projects (Jira-equivalent), with Git/GitHub commit history checked as evidence of individual contribution.

9.1 Work Breakdown Structure (WBS)

Work Package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements gathering, use cases, SRS baseline	All	4	SRS
WP2	High-level architecture + UML (use case, class, sequence, component)	Satvik L / Sourish Ganesh	4	Architecture / Design doc
WP3	Authentication & Presence module	Sourish Ganesh	6	Working feature
WP4	Direct Messaging module	Prithvi Aithal	6	Working feature
WP5	Group / Room Chat module	Satvik L	7	Working feature
WP6	Message history, status tracking, room administration	Prithvi Aithal / Satvik L	6	Working feature
WP7	Testing + CI/CD + security validation	All	6	Test / validation evidence
WP8	Final system validation + consolidated report	All	4	Final report

Total estimated effort: 43 person-days (~14 person-days per member) across a 12-week, six-sprint schedule.

9.2 Schedule (Sprint / Gantt Summary)

The table below is the text-form schedule; the team maintains the visual Gantt chart / burndown as a GitHub Projects board (screenshot attached as evidence in Deliverable 4).

Sprint	Weeks	Work Packages Active
Sprint 1	Weeks 1–2	WP1 (complete); WP2 (start)
Sprint 2	Weeks 3–4	WP2 (complete); WP3 (start)
Sprint 3	Weeks 5–6	WP3 (complete); WP4
Sprint 4	Weeks 7–8	WP5 (start); WP6 (start)
Sprint 5	Weeks 9–10	WP5, WP6 (complete); WP7
Sprint 6	Weeks 11–12	WP7 (complete); WP8

9.3 Tool Stack

Activity	Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects (Jira-equivalent)	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Workflow run screenshot/log
Containerization	Docker	Dockerfile + build/run evidence
Testing	Jest / Supertest (backend)	Test results
API testing	Postman	Collection / results
Static analysis	SonarQube or similar	Analysis report
Diagramming	draw.io / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
Socket / WebSocket	A persistent, full-duplex TCP-based connection that lets the server push events to a client without the client polling.
Room	A named group conversation that one or more registered users can join to exchange group messages.
Presence	A user's real-time online/offline connection state, broadcast to their contacts and rooms.
JWT	JSON Web Token – a signed token used to authenticate a user on REST calls and the WebSocket handshake.
Typing Indicator	A transient real-time event shown to a conversation's other participant(s) while a message is being composed.
Read Receipt	A status update shown to a sender indicating the recipient has viewed their message.
RBAC	Role-Based Access Control – restricting an action based on a user's assigned role (e.g. room admin vs member).
ODM	Object-Document Mapper – here, Mongoose, which maps MongoDB documents to JavaScript objects/schemas.

Appendix B: Field Layouts

Derived from the entity-relationship model in Figure 2, the core persisted fields are:

User

Field	Length	Data Type	Description	Mandatory
user_id	24	ObjectId/String	Unique user identifier (primary key)	Y
username	30	String	Unique display/login handle	Y
email	100	String	Unique email address	Y
password_hash	60	String	bcrypt hash of the user's password	Y
status	10	Enum	online / offline	Y

Room

Field	Length	Data Type	Description	Mandatory
room_id	24	ObjectId/String	Unique room identifier (primary key)	Y
name	50	String	Unique room name	Y
type	10	Enum	direct / group	Y
created_by	24	ObjectId/String	FK → User who created the room	Y

RoomMembership

Field	Length	Data Type	Description	Mandatory
membership_id	24	ObjectId/String	Unique membership identifier (primary key)	Y
room_id	24	ObjectId/String	FK → Room	Y
user_id	24	ObjectId/String	FK → User	Y
role	10	Enum	member / admin	Y

Message

Field	Length	Data Type	Description	Mandatory
message_id	24	ObjectId/String	Unique message identifier (primary key)	Y
room_id	24	ObjectId/String	FK → Room (direct conversations use a 2-member room)	Y
sender_id	24	ObjectId/String	FK → User who sent the message	Y
content	2000	String	Message text body	Y
timestamp	–	DateTime	Server-assigned send time	Y
status	10	Enum	sent / delivered / read	Y

Appendix C: Requirement Traceability Matrix

At the Requirements stage this matrix carries requirement IDs only; the Architecture, Design, Code and Test-Case reference columns will be completed once Deliverable 3 (Design) and Deliverable 4 (Implementation & Testing) are finalized, per the course's stage-wise validation-specification flow.

Sl. No	Requirement ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test Case ID
1	FR-001	Unique user registration	TBD	TBD	TBD	TBD	TBD
2	FR-007	Send direct message	TBD	TBD	TBD	TBD	TBD
3	FR-012	Create room	TBD	TBD	TBD	TBD	TBD
4	FR-014	Broadcast group	TBD	TBD	TBD	TBD	TBD

		message					
5	SEC-002	Room-admin RBAC	TBD	TBD	TBD	TBD	TBD

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Architecture & Design Specification

for

ChatSphere – Real-Time Direct & Group Messaging Platform

Version 1.0

Prepared by Team 16 – Satvik L, Prithvi Aithal, Sourish Ganesh
PES University, Bangalore – Dept. of CSE, Section G

September 2026

Revision History

Name	Date	Reason For Changes	Version
Team 16	Sep 2026	Initial Architecture & Design Specification baseline	1.0

Table of Contents

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of ChatSphere Version 1.0, building on the requirements baselined in the Software Requirements Specification (SRS-CHATSPHERE-16-1.1). It is the Level 3 Software Engineering mini-project (UE24CS341A) Deliverable 3 (Architecture & Design) for Team 16, and is the contract against which implementation (Deliverable 4) and design-level code review are performed.

1.2 Scope

This document covers: the system's architectural style and components (Section 2); the detailed design of representative interaction flows, the API contract and error-handling strategy (Section 3); and the traceability of both architecture and design elements back to SRS requirements (Section 4).

1.3 References

- ChatSphere Software Requirements Specification, Version 1.1 (SRS-CHATSPHERE-16-1.1).
- ChatSphere Test Plan, Version 1.0 (TP-CHATSPHERE-16-1.0).
- ChatSphere Synopsis / Project Proposal, Version 1.0.

2. Architecture

2.1 Architectural Pattern

ChatSphere follows a three-tier, event-driven layered architecture. The client tier (React SPA) communicates with the application tier over two channels: a stateless REST API (Express.js, used for registration, login and history retrieval) and a stateful WebSocket gateway (Socket.IO, used for every real-time event). The application tier is itself organised using a Service Layer pattern: each system feature from SRS Section 5 is implemented as an independent service (Auth & Presence, Messaging, Room) sitting behind a common JWT authentication middleware, which keeps business logic testable in isolation (per SRS quality attribute “Testability”) and lets each service be owned end-to-end by one team member (Synopsis Section 9). The persistence tier separates durable state (MongoDB, via Mongoose) from ephemeral/pub-sub state (Redis), reflecting the very different consistency and latency needs of chat history versus live presence.

This pattern was chosen over a monolithic single-file server because it lets each of the three team members build, unit-test and demo their owned feature independently within a sprint, and over a full microservices split (separate deployable processes per service) because the single-server deployment target for this phase (SRS Section 2.6) does not need independent scaling or deployment of each service yet — that migration path is kept open by the existing service-layer boundaries.

2.2 Component Diagram and Description

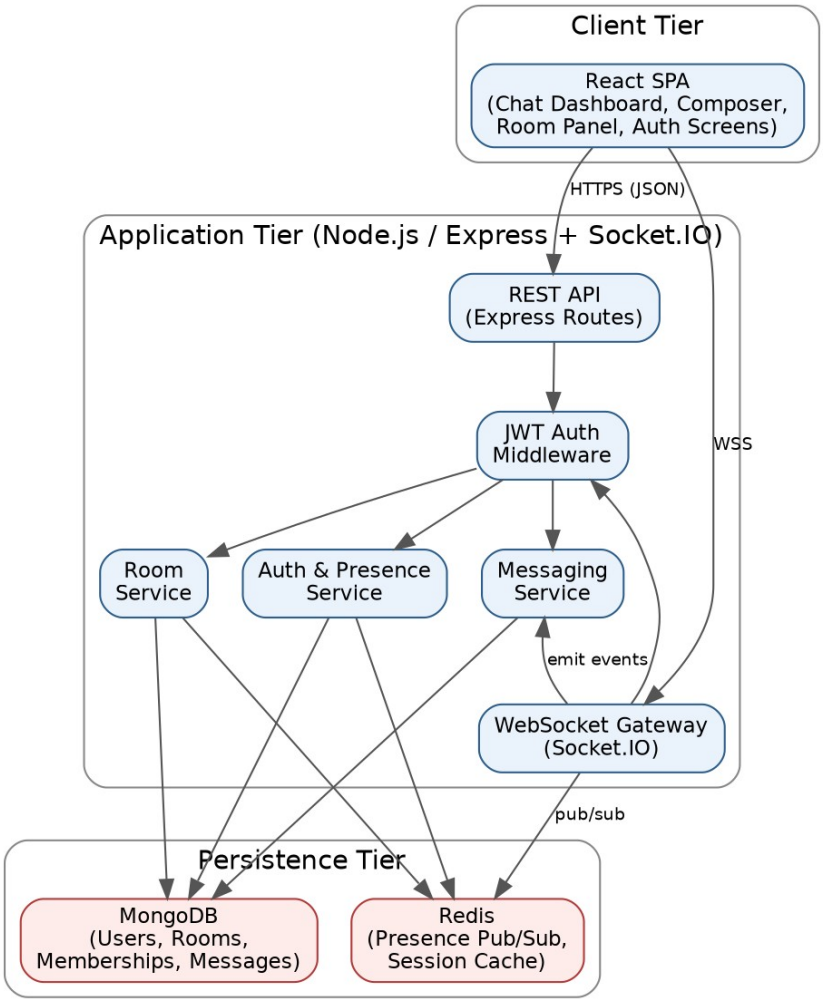


Figure 1 – Component Diagram for ChatSphere

Component responsibilities:

- React SPA — renders the Registration/Login screen, Chat Dashboard, Message Composer, Room Panel and Search (SRS Section 3.1); holds no business logic beyond client-side validation and UI state.
- REST API (Express Routes) — stateless HTTP endpoints for registration, login and paginated history retrieval (SRS Section 3.2).
- WebSocket Gateway (Socket.IO) — accepts the authenticated real-time connection and routes every socket event (message send, typing, presence, room join/leave, room-admin actions) to the appropriate service.
- JWT Auth Middleware — a single, shared component that verifies the JWT on every REST call and every WebSocket handshake/event before it reaches a service, so authentication logic is written and tested exactly once (supports SEC-004).
- Auth & Presence Service — owns registration, login, password hashing and online/offline presence broadcast (FR-001 to FR-006).

- Messaging Service — owns direct-message send/receive, typing indicators, delivery/read status and history persistence for 1:1 conversations (FR-007 to FR-011, FR-017, FR-018).
- Room Service — owns room create/join/leave, group-message broadcast and room administration (FR-012 to FR-016, FR-019 to FR-021).
- MongoDB — durable storage for User, Room, RoomMembership and Message documents (see SRS Appendix B for field layouts).
- Redis — presence pub/sub and the Socket.IO adapter, chosen to keep ephemeral, high-churn presence state out of the durable store and to leave a forward path to multi-instance scaling (SRS Section 2.6).

2.3 Traceability to Requirements

Component	SRS Requirements Satisfied
React SPA	SRS 3.1 (all UI requirements)
REST API	FR-001, FR-003, FR-018, SRS 3.2/3.3
WebSocket Gateway	FR-004, FR-005, FR-006, FR-007, FR-008, FR-010, FR-012 to FR-014, FR-019 to FR-021
JWT Auth Middleware	FR-004, SEC-001, SEC-004
Auth & Presence Service	FR-001 to FR-006, SEC-001
Messaging Service	FR-007 to FR-011, FR-017, FR-018, NFR-001
Room Service	FR-012 to FR-016, FR-019 to FR-021, SEC-002
MongoDB	FR-001, FR-012, FR-017, FR-018 (durable persistence)
Redis	FR-005, FR-006 (presence pub/sub)

2.4 Security Architecture

Security is designed as a set of layered controls that together satisfy the Security Objectives in SRS Section 6.3, rather than a single perimeter check:

- Transport security: all REST traffic is served over HTTPS and all real-time traffic over WSS (SRS Section 3.3), so credentials and messages are never sent in the clear (supports SEC-OBJ-1).
- Authentication: passwords are hashed with bcrypt before storage and never persisted in plaintext (SEC-001); a successful login issues a signed JWT, which is the only credential presented afterwards.
- Authorization (RBAC): the JWT Auth Middleware authenticates every REST call and WebSocket event; the Room Service additionally checks the requester's RoomMembership.role before any room-administration action, rejecting non-admins (SEC-002, FR-021) — illustrated in Sequence Diagram 2, Section 3.1.
- Input safety: all user-supplied message content is sanitized/escaped by the Messaging and Room Services before storage and again before the client renders it, to prevent stored XSS (SEC-003).
- Abuse resistance: the WebSocket Gateway rate-limits message-send events per connection (SEC-005).
- Auditability: failed logins and unauthorized room-admin attempts are logged by the Auth & Presence Service and Room Service respectively (SEC-006), so that the security tests in the Test Plan (Section 5.1) have an audit trail to check against.

2.5 Other Architectural Considerations

- Deployment view: the application tier is packaged as a single Docker image (Node.js 20 LTS on Linux); MongoDB and Redis run as separate containers in the same Docker Compose stack for this phase (SRS Section 2.4).
- Data architecture: the persisted schema (User, Room, RoomMembership, Message) follows the entity-relationship model in SRS Figure 2 and the field layouts in SRS Appendix B; RoomMembership is the join entity carrying each member's role, which is what the Security Architecture's RBAC check reads.
- Scalability path (explicitly out of scope for this phase, SRS Section 2.6): because presence state already lives in Redis and Socket.IO already supports a Redis adapter, moving to multiple application-tier instances later requires no change to the Messaging/Room service boundaries defined here.

3. Design

3.1 Sequence Diagrams

The two sequences below were chosen because they each exercise a different part of the Security Architecture (Section 2.4) alongside their core functional behaviour: one shows the normal, happy-path status lifecycle of a direct message; the other shows an authorization check rejecting an unauthorized action.

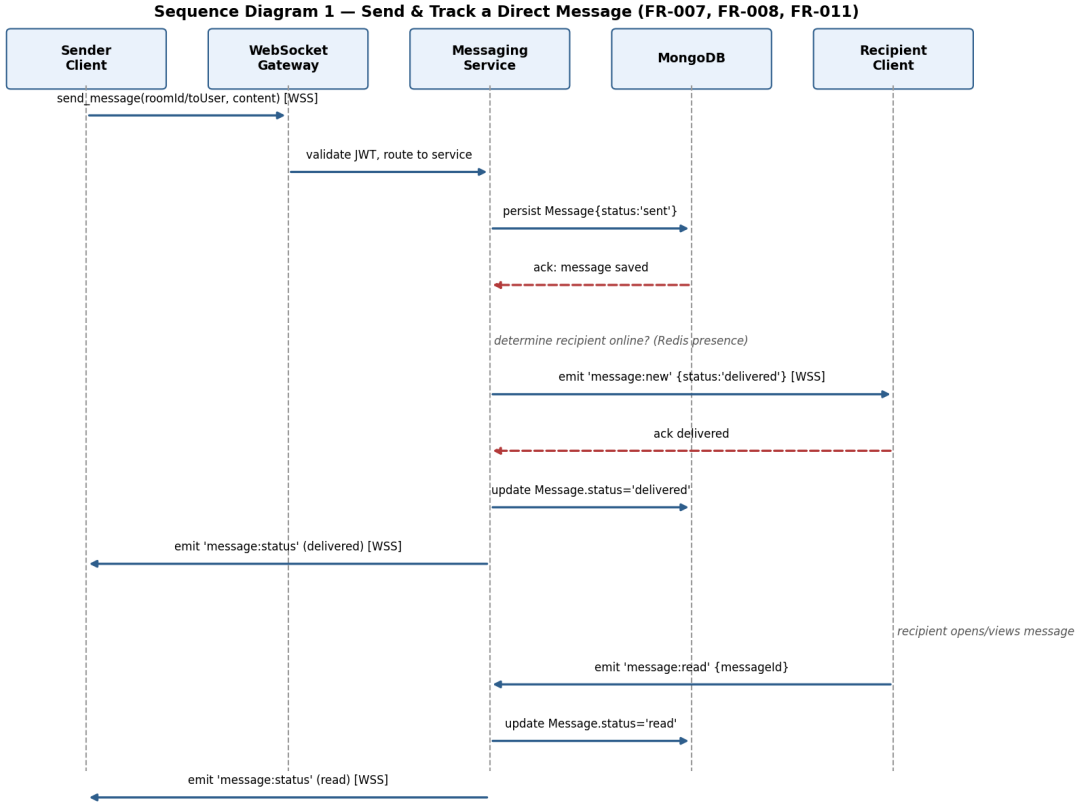


Figure 2 – Sequence Diagram 1: Send & Track a Direct Message (FR-007, FR-008, FR-011)

This flow shows the sender's message being persisted with status 'sent' before any delivery acknowledgement is sent back (SRS Section 6.2 data-safety requirement), then the Messaging Service checking Redis presence to decide whether to deliver immediately, and finally the status transition sent → delivered → read being pushed back to the sender in real time over the WebSocket Gateway.

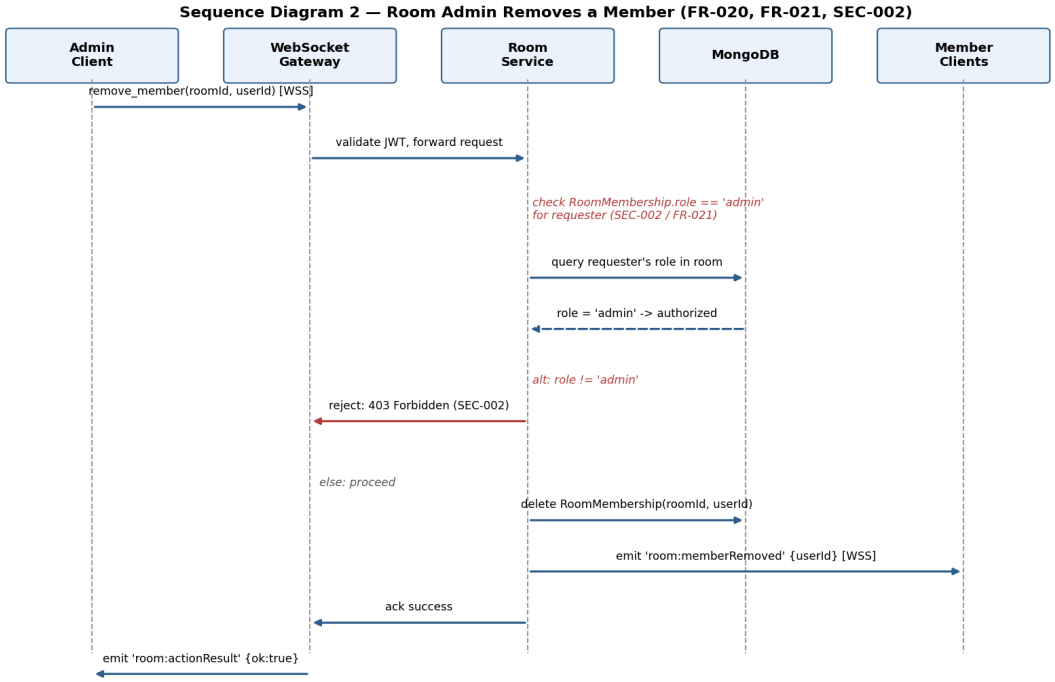


Figure 3 – Sequence Diagram 2: Room Admin Removes a Member (FR-020, FR-021, SEC-002)

This flow shows the Room Service checking the requester's RoomMembership.role before acting: the alt fragment shows the action rejected with an authorization error when the requester is not an admin, and only proceeding to delete the membership and broadcast the change when the requester is confirmed as admin.

3.2 API Design

3.2.1 REST API

Method & Path	Request Body	Response	Requirement(s)
POST /api/auth/register	{ username, email, password }	201 { userId } / 400 validation error	FR-001, FR-002, SEC-001
POST /api/auth/login	{ usernameOrEmail, password }	200 { token } / 401 invalid credentials	FR-003
GET /api/messages/:conversationId?page=&pageSize=	—	200 { messages[], nextPage } / 401 / 403	FR-018, SEC-004
GET /api/rooms/:roomId/messages?page=&pageSize=	—	200 { messages[], nextPage } / 401 / 403	FR-018, FR-015, SEC-004

All REST endpoints except /api/auth/register and /api/auth/login require a valid Bearer JWT (Authorization header); the JWT Auth Middleware returns 401 for a missing/expired/invalid token before the request reaches a service (SEC-004).

3.2.2 WebSocket (Socket.IO) Events

Event	Payload	Server→client	Requirement(s)

(client→server)		Event(s)	
message:send	{ toUserId | roomId, content }	message:new, message:status	FR-007, FR-008, FR-012 to FR-014
message:read	{ messageId }	message:status	FR-011
typing:start / typing:stop	{ conversationId | roomId }	typing:update	FR-010
room:create	{ name }	room:created	FR-012
room:join / room:leave	{ roomId }	room:memberJoined / room:memberLeft	FR-013
room:promote	{ roomId, userId }	room:memberPromoted / error	FR-019, SEC-002
room:removeMember	{ roomId, userId }	room:memberRemoved / error	FR-020, FR-021, SEC-002
(connect / disconnect)	JWT in handshake auth payload	presence:update	FR-004 to FR-006

3.3 Error Handling

Error handling follows one consistent convention across both the REST API and WebSocket events, so a single client-side error handler can cover both channels:

- REST errors return a JSON body { error: { code, message } } with the matching HTTP status (400 validation, 401 unauthenticated, 403 unauthorized, 404 not found, 429 rate-limited, 500 unexpected).
- WebSocket errors are emitted on a dedicated 'error' event with the same { code, message } shape, rather than silently dropping the client's request, so the UI can always show feedback (supports the Usability quality attribute in SRS Section 6.4).
- Authorization failures (SEC-002, SEC-004) always return code AUTH_FORBIDDEN / AUTH_INVALID_TOKEN and are logged server-side per SEC-006, without leaking whether a resource exists (e.g. a non-member is told the room action is forbidden, not that the room itself does not exist).
- Validation failures (e.g. FR-001 duplicate username, FR-016 removing the last admin) return code VALIDATION_FAILED with a field-level message and change nothing in the database (supports the Reliability quality attribute).
- Unexpected server errors are caught by a top-level Express error middleware and Socket.IO error boundary, logged with a correlation ID, and returned to the client as a generic INTERNAL_ERROR without internal detail.

3.4 Other Design Sections

3.4.1 Data Design

The persisted schema design reuses the entities already defined in SRS Appendix B (User, Room, RoomMembership, Message) unchanged; RoomMembership.role is the single source of truth read by the Room Service's authorization check in Sequence Diagram 2.

3.4.2 Concurrency and Idempotency

Because a client may retry a message send after a dropped connection (SRS reliability requirement, Section 6.4), each outgoing message carries a client-generated idempotency

key; the Messaging Service deduplicates on that key before persisting, so a retried send cannot create a duplicate message.

3.4.3 Testability Hooks

Each service exposes its socket event handlers as plain functions that take a parsed, already-authenticated payload, independent of the live Socket.IO transport, so the unit tests described in the Test Plan (Section 5.4) can call them directly with mocked sockets.

4. Design-Level Traceability Matrix

This matrix extends SRS Appendix C with the Architecture/Design references now available at this stage; Code and Test Case references remain TBD until Deliverable 4.

Requirement ID	Brief Description	Architecture Ref	Design Ref
FR-001	Unique user registration	Auth & Presence Service (2.2)	POST /api/auth/register (3.2.1)
FR-004	WebSocket JWT handshake check	JWT Auth Middleware (2.2, 2.4)	connect/disconnect events (3.2.2)
FR-007	Send direct message	Messaging Service (2.2)	message:send event; Sequence Diagram 1 (3.1)
FR-012	Create room	Room Service (2.2)	room:create event (3.2.2)
FR-014	Broadcast group message	Room Service, WebSocket Gateway (2.2)	message:send event (3.2.2)
FR-020	Admin removes a member	Room Service (2.2)	room:removeMember event; Sequence Diagram 2 (3.1)
SEC-002	RBAC on room-admin actions	Security Architecture (2.4)	Sequence Diagram 2 (3.1)
SEC-003	Sanitize message content (XSS)	Messaging/Room Service (2.4)	Error Handling – VALIDATION_FAILED (3.3)
SEC-004	Reject expired/invalid JWTs	JWT Auth Middleware (2.4)	REST/WebSocket auth errors (3.2, 3.3)

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Test Plan

for

ChatSphere – Real-Time Direct & Group Messaging Platform

Version 1.0

Prepared by Team 16 – Satvik L, Prithvi Aithal, Sourish Ganesh
PES University, Bangalore – Dept. of CSE, Section G

September 2026

Revision History

Name	Date	Reason For Changes	Version
Team 16	Sep 2026	Initial Test Plan baseline for Requirements submission	1.0

Table of Contents

1. Test Plan Identifier

Identifier: TP-CHATSPHERE-16-1.0. This Test Plan corresponds to Version 1.1 of the ChatSphere Software Requirements Specification (SRS-CHATSPHERE-16-1.1) and to the Level 3 Software Engineering mini-project (UE24CS341A) Deliverable 2 (Test Plan) for Team 16.

2. Introduction

2.1 Purpose

This Test Plan defines the scope, approach, resources and schedule of the testing activities for ChatSphere. It follows the structure of the IEEE 829 Standard for Software Test Documentation, adapted for the course's stage-wise deliverable flow: at this stage the plan is planning-focused (test items, features, approach, criteria, schedule and a representative set of test cases); concrete pass/fail execution results will be recorded in the System Validation Report once Deliverables 3 and 4 (Design, Implementation & Testing) are complete.

2.2 Scope

Testing covers every functional requirement (FR-001 to FR-021), non-functional requirement (NFR-001, NFR-002) and security requirement (SEC-001 to SEC-006) defined in the SRS, across the five system features: Authentication & Presence Management, Direct Messaging, Group/Room Chat, Message History & Status Tracking, and Room Administration. Items explicitly out of scope for this phase (end-to-end encryption, voice/video calling, file/media sharing, multi-server scaling, third-party SSO) per Synopsis Section 7 are excluded from this Test Plan.

2.3 References

- ChatSphere Software Requirements Specification, Version 1.1 (SRS-CHATSPHERE-16-1.1).
- ChatSphere Synopsis / Project Proposal, Version 1.0.
- IEEE Std 829-2008, Standard for Software and System Test Documentation.
- PES University – “Jackfruit – Software Engineering Project Guidelines 2026” (UE24CS341A).

3. Test Items

The following software items, as they will exist at the end of each relevant sprint, are subject to this Test Plan:

Item	Description	Version / Sprint
Auth & Presence Service	Registration, login (JWT issuance), WebSocket handshake authentication, presence broadcast	Sprint 3, v1.0
Direct Messaging Service	1:1 message send/receive, typing indicator, delivery/read status	Sprint 4, v1.0

Room / Group Chat Service	Room create/join/leave, group message broadcast	Sprint 5, v1.0
Message History Module	Persistence and paginated retrieval of direct & group messages	Sprint 5, v1.0
Room Administration Module	Promote/remove member, admin-role enforcement (RBAC)	Sprint 5, v1.0
REST API layer	Express.js endpoints for auth and history retrieval	Sprint 3-5, v1.0
WebSocket Gateway	Socket.IO event routing for all real-time features	Sprint 3-5, v1.0

4. Features to be Tested and Features Not to be Tested

4.1 Features to be Tested

Every functional, non-functional and security requirement in SRS Sections 5 and 6 is in scope for testing, summarized by feature:

- Authentication & Presence — FR-001 to FR-006, SEC-001, SEC-004.
- Direct Messaging — FR-007 to FR-011, NFR-001.
- Group / Room Chat — FR-012 to FR-016, SEC-002.
- Message History & Status Tracking — FR-017, FR-018, NFR-002.
- Room Administration — FR-019 to FR-021, SEC-002.
- Cross-cutting security — SEC-001 to SEC-006 (see Section 5.1).

4.2 Features Not to be Tested

The following are out of scope for this phase, per SRS Section 2.6 and Synopsis Section 7, and are therefore not tested:

- End-to-end message encryption.
- Voice / video calling.
- File, image or media sharing.
- Multi-instance / horizontal scaling of the Socket.IO layer.
- Third-party SSO (Google/GitHub login).
- Native mobile push notifications.

5. Approach

Testing follows a layered strategy: unit tests for isolated business logic, integration tests for service-to-service and service-to-database behaviour, functional (black-box) tests derived directly from each FR, and dedicated security and performance testing passes. Each test case in Appendix A is written against a specific requirement ID so results feed directly into the Requirement Traceability Matrix (Appendix B).

5.1 Security Validation Approach

Security validation is treated as a first-class, dedicated testing activity rather than a by-product of functional testing, in line with SEC-OBJ-1 to SEC-OBJ-4 in the SRS. It comprises:

- Authentication/authorization testing: verifying that REST endpoints and WebSocket connections reject missing, expired or tampered JWTs (SEC-004), and that room-administration actions are rejected for any user without the admin role for that room (SEC-002, FR-021).
- Negative/penetration-style testing: manually attempting room-admin REST/WebSocket calls with a non-admin JWT, an expired JWT and a JWT with a modified payload/signature; attempting to access a room's messages as a non-member (FR-015).
- Input-safety testing: submitting message content containing HTML/script payloads and confirming the content is sanitized/escaped before storage and before rendering, ruling out stored XSS (SEC-003).
- Configuration/code review: confirming, by inspection, that passwords are hashed with bcrypt and never persisted in plaintext (SEC-001) and that security-relevant events (failed logins, unauthorized admin attempts) are logged (SEC-006).
- Abuse/rate-limit testing: sending message-send events at a high rate from a single connection and confirming the rate limiter engages (SEC-005).

Security test results, including the penetration-style checks, will be consolidated into the System Validation Report at Deliverable 5, per SRS Section 8.

5.2 Functional Testing Approach

Black-box functional tests are derived one-to-one from the functional requirements in SRS Section 5. Each feature owner (per Synopsis Section 9) writes and executes the functional tests for their owned feature during the sprint in which it is built, with peer review of test cases before merge.

5.3 Non-Functional (Performance) Testing Approach

Latency and throughput targets (NFR-001, NFR-002) are validated with a scripted load-test client that opens multiple concurrent WebSocket connections and REST sessions, measuring the 95th-percentile message delivery time and REST response time against the defined test load.

5.4 Regression and Unit Testing Approach

Unit tests (Jest) cover business logic in each service (auth, messaging, room, presence) in isolation, using mocked sockets and a mocked/in-memory MongoDB instance. The full unit and integration suite is re-run automatically on every pull request via the GitHub Actions CI/CD pipeline, acting as the regression safety net for later sprints.

6. Item Pass/Fail Criteria

A test item (feature or requirement) passes when: (a) every test case tagged to it in Appendix A executes with the expected result; (b) no High-priority defect remains open against it; and (c) for NFR-001/NFR-002, the measured 95th-percentile value meets the stated threshold under the defined test load. A test case itself passes only when the actual

result exactly matches its expected result; any deviation is logged as a defect and the test case is marked Failed until re-verified.

7. Suspension Criteria and Resumption Requirements

- Testing of a feature is suspended if a blocking defect prevents further test cases for that feature from executing (e.g. the server fails to start, or the database connection cannot be established).
- Testing is suspended feature-wide if a High-severity security defect is found (e.g. a non-admin action is not rejected), until a fix is merged.
- Testing resumes once the blocking defect is fixed, the CI pipeline is green, and the affected test cases are re-run from the point of suspension.

8. Test Deliverables

- This Test Plan document (TP-CHATSPHERE-16-1.0).
- Test case specifications (Appendix A) and the Requirement Traceability Matrix (Appendix B).
- Automated test suite (Jest/Supertest) and CI run logs.
- Postman collection and results for REST API testing.
- Final System Validation Report (Deliverable 5), consolidating pass/fail results, defect log, and security/penetration-test findings.

9. Testing Tasks

Task	Description	Owner
Write test cases	Derive test cases from each FR/NFR/SEC requirement	Feature owner
Set up test environment	Docker-composed Node.js, MongoDB, Redis test stack; CI runner config	Sourish Ganesh
Implement unit/integration tests	Jest/Supertest suites per module	Feature owner
Execute functional tests	Run black-box tests against running server	All members
Execute security tests	Run authentication/authorization/negative/XSS/rate-limit checks	Sourish Ganesh
Execute performance tests	Run scripted load test against NFR-001/002	All members
Log and triage defects	Record, prioritize and assign defects found during testing	All members
Consolidate results	Produce the System Validation Report	All members

10. Environmental Needs

- Hardware/OS: Linux-based Docker host (development laptops + GitHub Actions runners).

- Software: Node.js 20 LTS, MongoDB 6+, Redis 7+, Jest, Supertest, Postman.
- Network: local Docker network for integration tests; HTTPS/WSS simulated with self-signed certificates in the test environment.
- Test data: a seeded set of test user accounts, rooms and messages created by a setup script before each test run and torn down after.

11. Responsibilities

Member	Testing Responsibility
Sourish Ganesh	Authentication & Presence tests; all security/penetration-style tests; CI/CD pipeline and test-environment setup
Prithvi Aithal	Direct Messaging and Message History tests; API/Postman test collections
Satvik L	Group/Room Chat and Room Administration tests; frontend-facing usability checks
All members	Unit tests for own owned feature; peer review of test cases; defect triage

12. Schedule

Testing is embedded in the six-sprint schedule defined in SRS Section 9.2, rather than run as a single end-of-project phase:

Sprint	Testing Activity
Sprint 1-2	Test Plan drafted; test environment and CI pipeline stood up
Sprint 3	Unit + functional tests for Authentication & Presence
Sprint 4	Unit + functional tests for Direct Messaging; usability walkthrough
Sprint 5	Unit + functional tests for Room Chat, History and Room Administration; full security and performance test pass
Sprint 6	Regression run, defect closure, System Validation Report

13. Risks and Contingencies

Risk	Contingency
A security defect (e.g. missing RBAC check) is found late in Sprint 5	Security tests for each module are run incrementally from Sprint 3 onward, not deferred to Sprint 5, to surface issues early
Load-test environment cannot reach the concurrency needed to validate NFR-001	Fall back to a scaled-down concurrency target with the shortfall documented in the System Validation Report
A team member's owned feature is delayed, compressing its test window	Cross-train a second member per module (per Synopsis Section 9) so testing can proceed in parallel

14. Approvals

Name	Role	Signature / Date
Satvik L	Team Member	
Prithvi Aithal	Team Member	
Sourish Ganesh	Team Member	
Course Faculty	Evaluator	

Appendix A: Test Cases

The following ten test cases give initial, representative coverage of functional, non-functional and security requirements. Additional test cases will be added as design and implementation proceed (Deliverables 3 and 4).

TC ID	Requirement(s)	Description	Preconditions	Steps	Expected Result
TC-01	FR-001	Register with unique username and email	Application reachable; username/email not already registered	1. Submit registration form with unique username, email, valid password.	Account is created; success response returned.
TC-02	FR-001 (negative)	Register with a duplicate email	An account already exists with the given email	1. Submit registration form re-using an existing email.	Registration is rejected with a clear validation error; no duplicate account is created.
TC-03	FR-003	Login with valid credentials	A registered account exists	1. Submit login form with correct username/email and password.	A signed JWT is returned and the client can open an authenticated session.
TC-04	FR-004, SEC-004	WebSocket handshake with expired JWT	A JWT has been issued and then allowed to expire	1. Attempt a Socket.IO connection using the expired token.	The server rejects the connection; no session is established.
TC-05	FR-008, NFR-001	Direct message delivered to online recipient	Sender and recipient are both authenticated and connected	1. Sender sends a direct message. 2. Measure time to delivery on recipient's client.	Message is received in real time; 95% of messages arrive within 300 ms under test load.
TC-06	FR-009	Direct message to offline recipient	Recipient is authenticated but not currently connected	1. Sender sends a direct message. 2. Recipient connects afterward.	Message is persisted while recipient is offline and delivered automatically on the recipient's next connection.

TC-07	FR-012, FR-014	Create room and broadcast group message	User is authenticated	1. Create a room with a unique name. 2. A second member joins. 3. Room creator sends a message.	Room is created with creator as first admin; message is broadcast in real time to every connected member.
TC-08	FR-021, SEC-002	Non-admin attempts to remove a room member	A room exists with a non-admin member and at least one other member	1. Non-admin member issues a remove-member action.	The action is rejected (e.g. 403 Forbidden); the target member is not removed.
TC-09	SEC-003	Message content sanitized against stored XSS	Sender is authenticated and a member of the target room	1. Send a message containing an embedded script tag.	The stored/rendered message is escaped/sanitized; the script does not execute in any client.
TC-10	FR-018	Retrieve paginated message history	A conversation/room with more messages than one page exists	1. Request message history for the conversation/room with a page size parameter.	History is returned in reverse-chronological order, correctly paginated.

Appendix B: Requirement Traceability Matrix

This matrix maps each SRS requirement to the test case(s) that verify it, or to the planned validation approach for requirements not yet covered by a listed test case (verified instead by code/configuration review, or by a test case to be added in a later sprint per SRS Section 8).

Requirement ID	Brief Description	Test Case(s) / Validation
FR-001	Unique user registration	TC-01, TC-02
FR-002	Passwords hashed with bcrypt	Code/configuration review (see SEC-001)
FR-003	Login issues signed JWT	TC-03
FR-004	WebSocket handshake JWT check	TC-04
FR-005	Broadcast online presence	Planned Sprint 3 integration test
FR-006	Broadcast offline presence within 5s	Planned Sprint 3 integration test
FR-007	Send direct message	TC-05
FR-008	Real-time delivery when online	TC-05
FR-009	Queue & deliver to offline recipient	TC-06
FR-010	Typing indicator	Planned Sprint 4 functional test
FR-011	Message status sent/delivered/read	TC-05, TC-06 (status observed)
FR-012	Create room, creator is first admin	TC-07
FR-013	Join / leave room	Planned Sprint 5 functional test
FR-014	Broadcast group message	TC-07
FR-015	Only members can view/send in room	Planned Sprint 5 negative/security test
FR-016	Room always retains ≥ 1 admin	Planned Sprint 5 negative test
FR-017	Persist every message	TC-05, TC-06 (persistence implied)
FR-018	Paginated history retrieval	TC-10
FR-019	Admin promotes a member	Planned Sprint 5 functional test
FR-020	Admin removes a member	TC-08 (authorized path to be added)
FR-021	Reject non-admin admin actions	TC-08
NFR-001	95% message delivery ≤ 300	TC-05

	ms	
NFR-002	95% REST response ≤ 1 s	Planned Sprint 5 performance test
SEC-001	No plaintext passwords	Code/configuration review
SEC-002	RBAC on room-admin actions	TC-08
SEC-003	Sanitize message content (XSS)	TC-09
SEC-004	Reject expired/invalid JWTs	TC-04
SEC-005	Rate-limit message-send events	Planned Sprint 5 load/security test
SEC-006	Log security-relevant events	Planned Sprint 5 system test / code review